

IHK – الگوی اصلی الگوریتم

هر الگو روی یک صفحه: شبه‌کد سبک IHK (مثل BubbleSort)، مثال جاوا، تحلیل فارسی و نکات. صفحه‌ی بعدی هر الگو: دو مثال واقعی/نزدیک به IHK با امضای متدهایی که در تمرین‌ها دیده‌ای.

Muster 1 – Suche / enthält (boolean)

جستجو در آرایه / لیست – پیدا کردن وجود یک عنصر

مسئلهٔ تیپیک: «بررسی کن آیا عنصری با این ویژگی وجود دارد یا نه»، خروجی معمولاً boolean است.

Pseudocode (IHK-Stil, مثل BubbleSort)

```
funktion enthaelt(feld: array<T>, gesucht: T) : boolean
    n = laenge(feld)
    für i = 0 bis n - 1
        wenn feld[i] = gesucht dann
            return true
        ende wenn
    ende für
    return false
end funktion
```

Java-Beispiel

```
public static <T> boolean enthaelt(T[] feld, T gesucht) {
    int n = feld.length;
    for (int i = 0; i < n; i++) {
        if (feld[i].equals(gesucht)) {
            return true;
        }
    }
    return false;
}
```

توضیح فارسی (تحلیل):

این الگو ساده‌ترین حالت جستجو است. ورودی یک آرایه یا لیست است و یک مقدار که دنبالش می‌گردیم. حلقه از اندیس ۰ تا آخر راه می‌رود و هر عنصر را با مقدار «gesucht» مقایسه می‌کند. به محض پیدا شدن، تابع با true برمی‌گردد. اگر حلقه تمام شد و چیزی پیدا نشد، جواب false است.

ترفند در امتحان: هر وقت امضای متدی را دیدی که خروجی‌اش boolean است و نامش چیزی مثل `enthaelt`, `existiert`, `hatTermin`, `istBelegt` بود، تقریباً همیشه همین الگو است: یک حلقه، یک if، و دو `return` (یکی در وسط، یکی در آخر).

نکتهٔ نمره‌ای: فراموش نکن بعد از حلقه حتماً یک `return false` بنویسی. خیلی‌ها در امتحان فقط داخل if مقدار true را برمی‌گردانند و `return` انتهایی را جا می‌اندازند.

دو مثال واقعی برای الگوی «جستجو»

Beispiel 1 – getAppointment(...): boolean

مسئله: بررسی شود آیا ترمینی با مشخصات داده‌شده (ماه، روز، slot، id) در آرایه ترمین‌ها وجود دارد یا نه.

```
funktion getAppointment(
    month: integer,
    day: integer,
    id: integer,
    slot: integer
) : boolean
n = laenge(termine)
für i = 0 bis n - 1
    wenn termine[i].monat = month
        und termine[i].tag = day
        und termine[i].id = id
        und termine[i].slot = slot dann
            return true
        ende wenn
    ende für
return false
end funktion
```

ایده: همان الگوی جستجو است فقط به‌جای یک شرط ساده، چند فیلد را با هم مقایسه می‌کند. هر ردیف اگر همه فیلدها برابر باشد یعنی ترمین پیدا شده است. **ترفند:** در امتحان هر وقت چند پارامتر (ماه، روز، شماره، ...) می‌بینی که باید با عنصر آرایه مقایسه شوند، به جستجوی چندشرطی فکر کن.

Beispiel 2 – sucheTopseller(kriterium, vorgabewert): String

مسئله: در لیست Artikel‌ها آن محصولی را پیدا کن که بر اساس یک معیار بیشترین مقدار را دارد و در عین حال حداقل از یک vorgabewert هم بزرگ‌تر است.

```
funktion sucheTopseller(
    kriterium: integer,
    vorgabewert: integer
) : string
besterWert = -999999
besteId = ""
n = laenge(artikelListe)

für i = 0 bis n - 1
    wert = ermittleWert(artikelListe[i], kriterium)
    wenn wert >= vorgabewert
        und wert > besterWert dann
            besterWert = wert
            besteId = artikelListe[i].nummer
        ende wenn
    ende für

return besteId
end funktion
```

اینجا همچنان یک جستجو است، ولی علاوه بر «وجود دارد یا نه»، باید «بهترین» را هم پیدا کند. بنابراین به‌جای خروجی boolean، خروجی Id بهترین رکورد است و یک متغیر besterWert نگه می‌داریم. نکته مهم این است که شرط «قبول» (بزرگ‌تر مساوی vorgabewert) و شرط «بهترین تا حالا» را با هم در if می‌گذاریم.

شمردن تعداد عناصر با یک شرط (Count)

مسئلهٔ تییپک: «چند روز قیمت بالا رفته؟ چند نفر Azubi هستند؟ چند ورودی معتبر داریم؟»

Pseudocode – Muster

```
funktion zaehleMitBedingung(feld: array<T>) : integer
    n = laenge(feld)
    anzahl = 0
    für i = 0 bis n - 1
        wenn <bedingung für feld[i]> dann
            anzahl = anzahl + 1
        ende wenn
    ende für
    return anzahl
end funktion
```

Java-Beispiel (allgemein)

```
public static <T> int zaehleMitBedingung(T[] feld) {
    int n = feld.length;
    int anzahl = 0;
    for (int i = 0; i < n; i++) {
        if (/* Bedingung auf feld[i] */) {
            anzahl++;
        }
    }
    return anzahl;
}
```

تحلیل: یک شمارنده داریم که از ۰ شروع می‌شود و روی هر عنصر بررسی می‌کنیم آیا شرط برقرار است یا نه. اگر بله، یک واحد به شمارنده اضافه می‌کنیم. در آخر، شمارنده را برمی‌گردانیم. هر جا سوال می‌پرسد «چند تا» یا «۹۰٪»، «... Anzahl der ...» مواقع این الگو است.

در امتحان اگر در صورت سؤال کلماتی مثل **Anzahl, zählen, wie viele** دیدی، مستقیم به این اسکلت فکر کن: مقدار اولیه ۰، حلقه، if، افزایش.

دو مثال برای الگوی «شمارش با شرط»

Beispiel 1 – notierungPlus(Tageskurs[] kurse): Integer

مسئله واقعی: در جدول Kurs ها بشمار چند روز قیمت امروز از قیمت دیروز بیشتر بوده است.

```
funktion notierungPlus(kurse: array<Tageskurs>) : integer
    n = laenge(kurse)
    anzahl = 0
    für i = 1 bis n - 1
        wenn kurse[i].wert > kurse[i - 1].wert dann
            anzahl = anzahl + 1
        ende wenn
    ende für
    return anzahl
end funktion
```

اینجا شرط ما «Kurs امروز < Kurs دیروز» است. دقت کن حلقه از ۱ شروع می‌شود چون برای i=1 باید عنصر قبلی وجود داشته باشد. این نوع «مقایسه با عنصر قبلی» هم در امتحان زیاد تکرار می‌شود.

Beispiel 2 – erstelle_liste(stundensatz: double)

مسئله تکراری IHK: لیستی بساز از تمام Mitarbeiter هایی که بر اساس ساعت کاری و stundensatz به حد مشخصی می‌رسند یا از آن می‌گذرند.

```
funktion erstelle_liste(stundensatz: double)
    : liste<Mitarbeiter>

    liste = neueListe()
    n = laenge(mitarbeiterArray)

    für i = 0 bis n - 1
        lohn = mitarbeiterArray[i].stunden * stundensatz
        wenn lohn >= mindestLohn dann
            liste.hinzufuegen(mitarbeiterArray[i])
        ende wenn
    ende für

    return liste
end funktion
```

این مثال دقیقاً همان الگوی شمارش است با این تفاوت که به‌جای اینکه فقط تعداد را بشمارد، آن‌هایی را که شرط را دارند در یک Liste جمع می‌کند. از نظر ساختار حلقه و if همان ترفند است.

حداقل، حداکثر، میانگین و طولانی‌ترین دنباله

مسائل کلاسیک: پیدا کردن کوچک‌ترین / بزرگ‌ترین مقدار، محاسبهٔ متوسط، یا طولانی‌ترین دنباله‌ای که شرطی را ارضا می‌کند.

Min / Max – Pseudocode

```
funktion findeMin(werte: array<integer>) : integer
    n = laenge(werte)
    min = werte[0]
    für i = 1 bis n - 1
        wenn werte[i] < min dann
            min = werte[i]
        ende wenn
    ende für
    return min
end funktion
```

```
funktion findeMax(werte: array<integer>) : integer
    n = laenge(werte)
    max = werte[0]
    für i = 1 bis n - 1
        wenn werte[i] > max dann
            max = werte[i]
        ende wenn
    ende für
    return max
end funktion
```

Mittelwert – Pseudocode

```
funktion mittelwert(werte: array<integer>) : double
    n = laenge(werte)
    summe = 0
    für i = 0 bis n - 1
        summe = summe + werte[i]
    ende für
    return summe / n
end funktion
```

تحلیل: الگوی Min/Max یک متغیر نگه‌دارنده دارد (min/max) که ابتدا با اولین عنصر مقداردهی می‌شود، بعد در هر قدم اگر مقدار جدید بهتر بود جایگزین می‌شود. برای طولانی‌ترین دنباله (مثل maxPeriod) دو متغیر داریم: طول فعلی و طول بهترین تا حالا.

هر جا در صورت سؤال «durchschnittliche», «niedrigste», «höchste», یا «längste Periode» دیدی، مستقیم به این اسکلت‌ها فکر کن.

دو مثال: maxPeriod و متوسط مصرف

Beispiel 1 – maxPeriod(sensor_id, mindestwert): Integer

مسئله: در لیست مقادیر سنسور، طولانی‌ترین دنباله‌ای را پیدا کن که مقدار سنسور برای یک sensor_id مشخص از حداقل mindestwert کم نباشد.

```
funktion maxPeriod(sensor_id: integer,
                    mindestwert: double) : integer
    aktuelleLaenge = 0
    bestelaenge = 0
    n = laenge(messwerte)

    für i = 0 bis n - 1
        wenn messwerte[i].id = sensor_id
            und messwerte[i].wert >= mindestwert dann
                aktuelleLaenge = aktuelleLaenge + 1
                wenn aktuelleLaenge > bestelaenge dann
                    bestelaenge = aktuelleLaenge
            ende wenn
        sonst
            aktuelleLaenge = 0
        ende wenn
    ende für

    return bestelaenge
end funktion
```

این همان الگوی «بزرگ‌ترین دنباله متوالی» است: هر وقت شرط برقرار است، طول دنباله فعلی را یک واحد زیاد می‌کنیم؛ اگر شکسته شد، طول فعلی ۰ می‌شود. در هر قدم بهترین طول تا حالا را هم نگه می‌داریم.

Beispiel 2 – متوسط مصرف از آرایه

این نوع سؤال در مصرف انرژی، نمره‌ها، ساعات کار و غیره تکرار می‌شود.

```
funktion durchschnittVerbrauch(
    werte: array<integer>
) : double
    n = laenge(werte)
    summe = 0
    für i = 0 bis n - 1
        summe = summe + werte[i]
    ende für
    return summe / n
end funktion
```

فقط دقت کن نوع خروجی double باشد که تقسیم اعشاری درست انجام شود. در جاوا باید یکی از دو عدد به double تبدیل شود.

محاسبه رقم كنترلی و تبدیل عدد دهدهی → باینری

دو نوع سؤال تکراری IHK: Prüfwörter (وزندهی و mod) و Umrechnung (تقسیم و باقیمانده).

Prüfwörter – Pseudocode Muster

```
funktion pruefwort(ziffern: array<integer>,
                  gewichte: array<integer>) : integer
    summe = 0
    n = laenge(ziffern)
    g = laenge(gewichte)

    für i = 0 bis n - 1
        summe = summe + ziffern[i] * gewichte[i mod g]
    ende für

    return summe mod 10
end funktion
```

Umrechnung dezimal → binär

```
funktion dezimalZuBinär(n: integer) : string
    wenn n = 0 dann
        return "0"
    ende wenn

    ergebnis = ""
    solange n > 0
        rest = n mod 2
        ergebnis = konvertiere(rest) + ergebnis
        n = n / 2
    ende solange

    return ergebnis
end funktion
```

تحلیل Prüfwörter: آرایه ارقام را با آرایه وزنها ضرب می‌کنی و جمع می‌کنی. وزنها اگر تمام شدند، با $i \bmod g$ دوباره از اول شروع می‌شوند. آخر کار معمولاً $\bmod 10$ یا $\bmod 11$ گرفته می‌شود.

تحلیل Umrechnung: تقسیم پی‌درپی بر مبنای جدید (مثلاً ۲) و جمع کردن باقیمانده‌ها از راست به چپ.

دو مثال: Umrechnung(dezimalzahl) و ermittlePrüfziffer(Integer[])

Beispiel 1 – ermittlePrüfziffer(Integer[])

```
funktion ermittlePruefziffer(ziffern: array<integer>)
    : integer

    gewichte = [2, 3, 4, 5, 6, 7]
    summe = 0
    n = laenge(ziffern)
    g = laenge(gewichte)

    für i = 0 bis n - 1
        summe = summe + ziffern[i] * gewichte[i mod g]
    ende für

    return summe mod 10
end funktion
```

این دقیقاً همان الگوی عمومی است با آرایه وزن ثابت. در امتحان فقط اعداد وزن و مدول (۱۰ یا ۱۱) عوض می‌شوند.

Beispiel 2 – void Umrechnung(dezimalzahl: int)

```
prozedur Umrechnung(dezimalzahl: integer)
    wenn dezimalzahl = 0 dann
        ausgabe "0"
        return
    ende wenn

    ergebnis = ""
    n = dezimalzahl

    solange n > 0
        rest = n mod 2
        ergebnis = konvertiere(rest) + ergebnis
        n = n / 2
    ende solange

    ausgabe ergebnis
end prozedur
```

فرق تابع و پروسیدر در اینجاست: این‌جا چیزی برنمی‌گردانیم و فقط چاپ می‌کنیم، بنابراین prozedur است. در جاوا معادلش یک متد void است که System.out.println استفاده می‌کند.

مرتب‌سازی سادهٔ آرایه براساس یک Feld (z. B. Raum-Nummer)

BubbleSort ساده روی آرایه‌ای از اشیاء، بدون Comparator خارجی، دو حلقه و تعویض عناصر.

Pseudocode – Muster

```

prozedur sortiereNachFeld(feld: array<T>)
  n = laenge(feld)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn feld[j] > feld[j + 1] dann
        tausche feld[j], feld[j + 1]
      ende wenn
    ende für
  ende für
end prozedur

```

Java-Beispiel (BubbleSort auf int[])

```

static void bubbleSort(int[] a) {
  int n = a.length;
  for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++) {
      if (a[j] > a[j + 1]) {
        int tmp = a[j];
        a[j] = a[j + 1];
        a[j + 1] = tmp;
      }
    }
  }
}

```

الگوی مرتب‌سازی ساده همیشه دو حلقه دارد. شرط درونی «اگر عنصر بعدی باید جلوتر باشد، جابه‌جا کن» است. برای آرایهٔ اشیاء، در شرط به‌جای مقایسهٔ خود شیء معمولاً فیلدش را مقایسه می‌کنیم (مثل `raeume[j].nummer`).

دو مثال: sortKurseNachDatum و sort(raeume)

Beispiel 1 – void sort(raeume: Array von Raum)

```

prozedur sort(raeume: array<Raum>)
  n = laenge(raeume)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn raeume[j].nummer > raeume[j + 1].nummer dann
        tausche raeume[j], raeume[j + 1]
      ende wenn
    ende für
  ende für
end prozedur

```

همان BubbleSort است با این تفاوت که فیلد مقایسه nummer است، در جاوا شرط تبدیل می‌شود به `raeume[j].getNummer() > raeume[j+1].getNummer()`.

Beispiel 2 – sortiere Kurse nach Datum

```

prozedur sortiereKurseNachDatum(kurse: array<Tageskurs>)
  n = laenge(kurse)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn kurse[j].datum > kurse[j + 1].datum dann
        tausche kurse[j], kurse[j + 1]
      ende wenn
    ende für
  ende für
end prozedur

```

فقط فیلد مقایسه عوض شده (datum)، ساختار حلقه‌ها و عمل swap دقیقاً همان است، در بسیاری از امتحان‌ها از همین ساختار برای مرتب‌سازی Kunden, Artikel, Räume استفاده می‌شود.

مرتب‌سازی با تابع مقایسه (Comparator / vergleicheXY)

IHK گاهی sort را عمومی می‌کند و تابع مقایسه را جدا به عنوان پارامتر می‌دهد (مثل `(sort(List<T> liste, Function<T> f)`).

Pseudocode – sort mit comparator

```

prozedur sort(liste: array<T>,
              cmp: funktion(T, T): integer)
  n = laenge(liste)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn cmp(liste[j], liste[j + 1]) > 0 dann
        tausche liste[j], liste[j + 1]
      ende wenn
    ende für
  ende für
end prozedur

```

Beispiel Comparator – vergleicheMitarbeiter

```

funktion vergleicheMitarbeiter(
  m1: Mitarbeiter,
  m2: Mitarbeiter
) : integer
  wenn m1.name < m2.name dann
    return -1
  sonst wenn m1.name > m2.name dann
    return 1
  sonst
    wenn m1.id < m2.id dann
      return -1
    sonst wenn m1.id > m2.id dann
      return 1
    sonst
      return 0
    ende wenn
  ende wenn
end funktion

```

تابع مقایسه همیشه سه خروجی ممکن دارد: منفی (اولی کوچک‌تر است)، مثبت (اولی بزرگ‌تر است)، یا صفر (برابرند). در جاوا این تابع یا به صورت `compareTo` پیاده می‌شود یا یک `Comparator`.

دو مثال: sort(Tageskurs[], Function) و vergleicheMitarbeiter

Beispiel 1 – vergleicheMitarbeiter(m1, m2): Integer

در بسیاری از امتحان‌ها مقایسهٔ اشیاء چندمرحله‌ای است: اول بر اساس نام، اگر برابر بود بر اساس id.

```
funktion vergleicheMitarbeiter(
    m1: Mitarbeiter,
    m2: Mitarbeiter
) : integer
wenn m1.name < m2.name dann
    return -1
sonst wenn m1.name > m2.name dann
    return 1
sonst
    wenn m1.id < m2.id dann
        return -1
    sonst wenn m1.id > m2.id dann
        return 1
    sonst
        return 0
    ende wenn
ende wenn
end funktion
```

Beispiel 2 – sort(Tageskurs[] kurse, Function vergleiche)

```
prozedur sort(
    kurse: array<Tageskurs>,
    vergleiche: funktion(Tageskurs, Tageskurs): integer
)
n = laenge(kurse)
für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
        wenn vergleiche(kurse[j], kurse[j + 1]) > 0 dann
            tausche kurse[j], kurse[j + 1]
        ende wenn
    ende für
ende für
end prozedur
```

اینجا تابع `vergleiche` بیرون نوشته شده و به `sort` داده می‌شود. در صورت سؤال معمولاً امضایی مثل `sort(List<T> liste, Function<T> f)` می‌بینی. ترفند این است که در `if` به جای مقایسهٔ مستقیم، از خروجی `vergleiche` استفاده کنی.

کار با آرایهٔ دوبعدی ([][]int) + لیست نتایج خاص

این الگو در سؤال‌هایی مثل مصرف سالانه (۱۲ ماه $\times$ n Verbraucher) یا جدول بازدیدکنندگان (روز $\times$ ساعت) ظاهر می‌شود.

Pseudocode – Statistik mit Limit

```
funktion statistik2D(feld: array<array<integer>>,
                    limit: integer) : statistik
    min = +unendlich
    max = -unendlich
    liste = neuereListe() // z. B. liste<integer>

    zeilen = laenge(feld)

    für i = 0 bis zeilen - 1
        hatLimit = false
        spalten = laenge(feld[i])

        für j = 0 bis spalten - 1
            wert = feld[i][j]

            wenn wert < min dann
                min = wert
            ende wenn
            wenn wert > max dann
                max = wert
            ende wenn
            wenn wert > limit dann
                hatLimit = true
            ende wenn
        ende für

        wenn hatLimit dann
            liste.hinzufuegen(i)
        ende wenn
    ende für

    return neueStatistik(min, max, liste)
end funktion
```

دو حلقه تو در تو (i برای Zeile، j برای Spalte) داریم. درون حلقهٔ داخلی همزمان min/max را آپدیت می‌کنیم و یک پرچم (hatLimit) برای هر سطر نگه می‌داریم. بعد از تمام شدن هر سطر، اگر شرط برقرار بود، شمارهٔ آن سطر را در لیست خاص اضافه می‌کنیم.

دو مثال: statistik(verbrauch[][]), countVisitors(...)**Beispiel 1 – statistik(verbrauch: int[][], limit: int)**

```

funktion statistik(
    verbrauch: array<array<integer>>,
    limit: integer
) : jahresstatistik
min = +unendlich
max = -unendlich
liste = neueListe()

anzahlVerbraucher = laenge(verbrauch)

für v = 0 bis anzahlVerbraucher - 1
    hatLimit = false
    monate = laenge(verbrauch[v])

    für m = 0 bis monate - 1
        wert = verbrauch[v][m]

        wenn wert < min dann
            min = wert
        ende wenn
        wenn wert > max dann
            max = wert
        ende wenn
        wenn wert > limit dann
            hatLimit = true
        ende wenn
    ende für

    wenn hatLimit dann
        liste.hinzufuegen(v)
    ende wenn
ende für

return neueJahresstatistik(min, max, liste)
end funktion

```

Beispiel 2 – countVisitors(entry: ComeLeave): Integer

مسئله: از لیست ورود/خروج بازدیدکنندگان، یک جدول دو بعدی (روز × ساعت) بساز که نشان دهد در هر سلول چند نفر حاضر هستند.

```

funktion countVisitors(
    eintraege: liste<ComeLeave>,
    tage: integer,
    stunden: integer
) : array<array<integer>>
tabelle = neuesArray2D(tage, stunden, 0)

für jeden e in eintraege
    tag = e.tag
    stunde = e.stunde

    wenn e.typ = "KOMMEN" dann
        tabelle[tag][stunde] =
            tabelle[tag][stunde] + 1
    sonst
        tabelle[tag][stunde] =

```

```
        tabelle[tag][stunde] - 1
    ende wenn
ende für

return tabelle
end funktion
```

اینجا هم ساختار دوبعدی است اما به جای حلقه تو در تو، جدول را از روی یک لیست Ereignis ها پر می کنیم. هر رویداد یک سلول را یک واحد بالا یا پایین می برد.
الگو: ساختن Matrix با مقدار اولیه صفر و آپدیت در حلقه تک بعدی.